DMFR-YOLO: Defect Morphology Encoding and Feature Reassembly for Fabric Defect Detection

Kangzhao Tan, Jiarui Zhang, Junjie Yu, Heng Wu, and Chunhua He

***Abstract*—Circular knitting machines are widely used in continuous fabric production. Needle or yarn abnormalities can cause repeated defects along the fabric before they are detected, resulting in material waste and economic losses. To enable in-process defect detection and intervention, we develop a closed-loop defect detection system with edge inference and wired machine-stop control, and propose a lightweight detector termed DMFR-YOLO. DMFR-YOLO introduces defect morphology encoding (DME) into the backbone to explicitly model horizontally elongated defects, vertically elongated defects, and localized hole-like defects. In the feature pyramid, content-adaptive feature reassembly (CAFR) replaces fixed upsampling with content-adaptive reconstruction, improving the preservation of defect boundaries and weak responses during multi-scale feature fusion. Evaluated on a self-collected fabric defect dataset from a real-world circular knitting machine production line, DMFR-YOLO achieves 66.3% AP and 96.9% AP50, improving the YOLOv8n baseline by 7.0 and 2.8 percentage points, respectively, with only 3.39 M parameters. The detector is deployed as an INT8 RKNN model on an RK3566 edge device. The resulting system couples real-time defect detection, automatic local stop control, operator review, and cloud-based quality analysis, providing a practical solution for reducing defect propagation during circular-knitting production.**

***Index Terms*—Industrial defect detection, YOLO, deep learning, edge computing, Internet of Things.**

I. Introduction

CIRCULAR knitting machines are widely used in weft-knit fabric production and support high-rate, large-scale manufacturing [1], [2]. The resulting fabrics are used in everyday garments such as T-shirts and sportswear [2], [3]. The International Textile Manufacturers Federation reported global shipments of approximately 24,500 large circular knitting machines in 2025 [4]. Conventional quality checks are often performed after knitting is completed. Persistent needle or yarn abnormalities may therefore generate repeated defects along the fabric before detection, resulting in fabric damage and economic losses. [5]–[7]. This motivates in-process defect detection to enable timely intervention and limit defect propagation during continuous production.

Single-stage YOLO detectors combine rapid inference with explicit bounding-box localization [8], making them suitable for industrial defect detection. YOLO-based approaches have been applied to steel defects and large stamped parts [9]–[11]. For fabrics, existing designs mainly pursue lightweight architectures for efficient inference [12], [13], adaptive or deformable feature extraction for irregular defects [14]–[16], and improved multi-scale fusion or attention for small and weak defects [17]–[22]. Together, these studies demonstrate the suitability of YOLO-based detectors for fabric defect detection. However, these improvements mainly focus on detector efficiency or general feature enhancement, with less attention paid to the specific spatial characteristics of fabric defects.

In circular-knitted fabrics, the three defect categories considered in this work exhibit distinct spatial morphologies: horizontally elongated defects, vertically elongated defects, and localized hole-like defects. Conventional feature extraction can learn these patterns, but it does not explicitly encode their spatial characteristics. In addition, nearest-neighbor upsampling enlarges feature maps through fixed replication without adapting to local feature content. This may limit the preservation of thin defect boundaries and weak responses during multi-scale feature fusion.

Motivated by these characteristics, we develop DMFR-YOLO by modifying feature encoding and feature reconstruction. Defect morphology encoding (DME) introduces horizontal, vertical, and local branches at selected backbone stages to strengthen the representation of the three characteristic defect morphologies. Content-adaptive feature reassembly (CAFR) predicts reassembly weights from local feature content and replaces fixed top-down feature-pyramid upsampling with location-adaptive neighborhood aggregation. Together, the two modules encode the spatial forms of defects in the backbone and adaptively reconstruct these features across pyramid scales.

To apply these detector improvements in production, the model must operate in coordination with the machine and support local intervention. We deploy DMFR-YOLO on an RK3566 edge device so that defect decisions do not depend on a cloud round trip [23]. Machine rotation detected by the Hall sensor activates local detection. When a defect is detected, the system issues a wired stop command and displays the detection result for operator review. The IoT cloud receives defect and device records for monitoring and production-quality analysis, but does not participate in the stop decision. This integration enables the proposed detector to support real-time defect detection and timely intervention during circular-knitting production.

The main contributions of our work are as follows.

- We introduce DME to encode the characteristic morphologies of horizontal defects, vertical defects, and holes through horizontal, vertical, and local branches, providing a lightweight morphology-aware representation for circular-knitted defect detection.

Kangzhao Tan, Jiarui Zhang, Junjie Yu, and Chunhua He are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China (e-mail: tankangzhao@mails.gdut.edu.cn; zhangjiarui1@mail2.gdut.edu.cn; yujunjie2@mails.gdut.edu.cn; hechunhua@pku.edu.cn).

Heng Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mail: heng.wu@foxmail.com).

Corresponding author: Chunhua He.

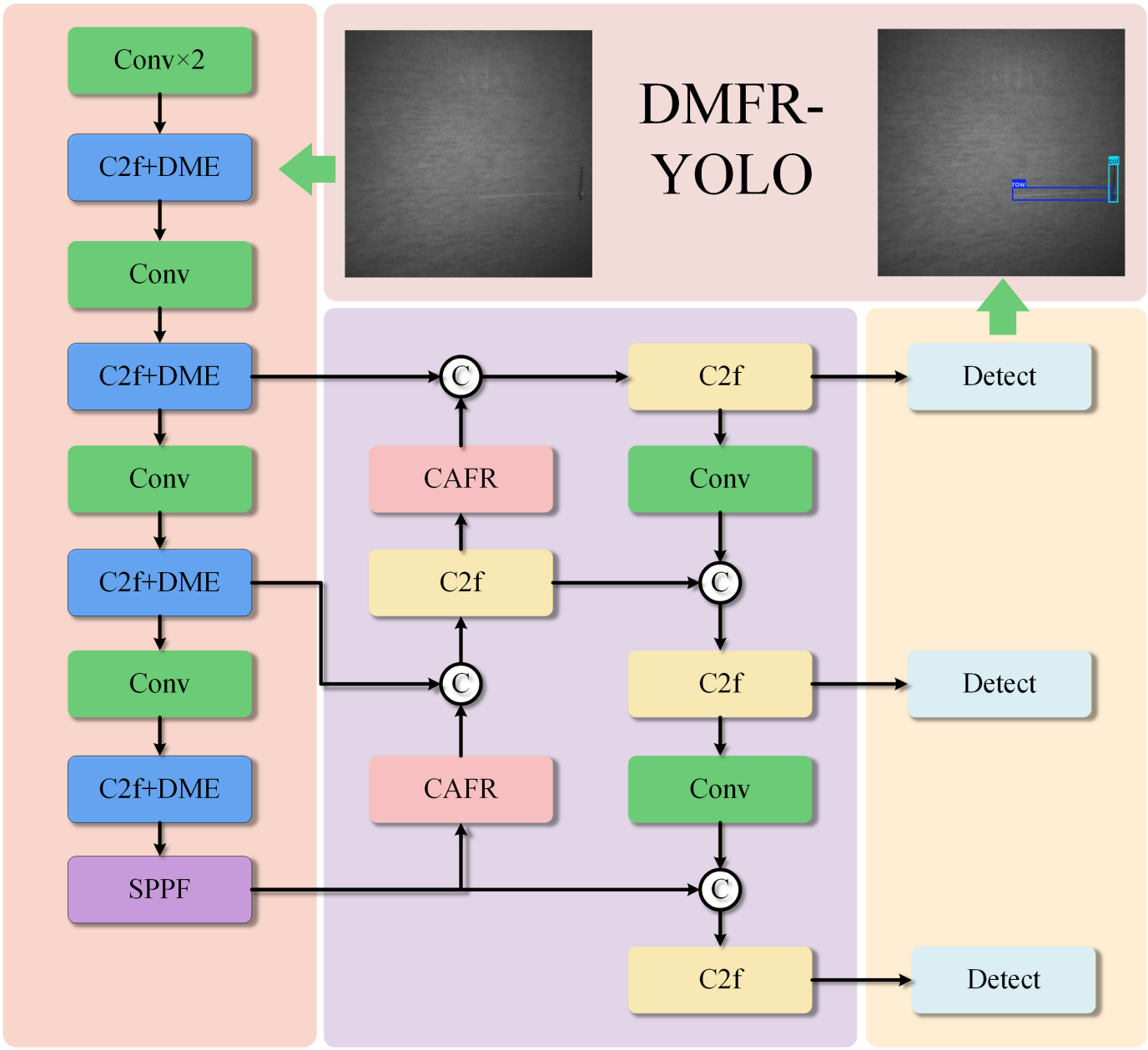

Fig. 1. Architecture of DMFR-YOLO. DME follows the backbone C2f stages, and CAFR replaces the two top-down upsampling operations. The neck fusion blocks, bottom-up path, and three-scale detection head are retained from YOLOv8n.

- We incorporate CAFR into the feature pyramid to replace fixed upsampling with locally predicted reassembly weights, enabling content-adaptive reconstruction during multi-scale fusion.
- We implement a closed-loop production-line defect detection system integrating RK3566 edge inference, wired machine-stop control, operator review, and cloud-based monitoring, and demonstrate its deployment on a real circular-knitting production line.

II. Proposed Method

A. DMFR-YOLO Architecture

DMFR-YOLO is developed based on YOLOv8n, which is selected for its compatibility with RK3566 deployment, compact model size, and modular architecture [24], [25]. The proposed modifications focus on defect morphology encoding in the backbone and feature reconstruction in the feature pyramid.

Fig. 1 shows the overall architecture. DME is introduced after the main backbone stages to enhance the representation of different defect morphologies. The extracted multi-scale features are then passed to the neck for feature fusion [26], [27]. Within the feature pyramid, CAFR replaces two nearest-neighbor upsampling operations with content-adaptive feature reassembly to better preserve defect boundaries and weak responses. The fused features are subsequently used by the multi-scale detection head to predict defect locations and categories.

B. Defect Morphology Encoding

The three defect categories considered in this work present different spatial characteristics. Horizontal and vertical defects usually extend along one dominant direction, whereas hole defects are more localized. To capture these differences, we introduce a defect morphology encoding (DME) module with three parallel branches, as shown in Fig. 2. The horizontal and vertical branches use 1×7 and 7×1 depthwise convolutions to enlarge the receptive field along the corresponding spatial directions, while the 3×3 branch captures localized defect structures. Depthwise convolution is adopted to model these spatial patterns with low computational overhead before channel-wise feature fusion [28].

For a C2f output $X \in \mathbb{R}^{C\times H\times W}$, the three branch responses are

$$F_h = \mathrm{DW}_{1\times 7}(X), \tag{1}$$
$$F_v = \mathrm{DW}_{7\times 1}(X), \tag{2}$$
$$F_l = \mathrm{DW}_{3\times 3}(X). \tag{3}$$

Here, DW denotes depthwise convolution followed by batch normalization and SiLU activation. All branches preserve the spatial dimensions and produce C output channels. Their concatenation is fused as

$$F_d = \phi\left([F_h, F_v, F_l]\right), \tag{4}$$

where $[\cdot]$ denotes channel concatenation and ϕ is a 1×1 convolution mapping $3C$ channels to C, followed by batch normalization and SiLU. This learnable fusion combines the three types of spatial evidence rather than assigning a fixed branch to a particular defect class. A residual connection [29] gives

$$Y = X + F_d. \tag{5}$$

The residual connection combines the morphology-aware features with the incoming C2f output, allowing DME to enhance the backbone representation without discarding the features learned by the preceding layers. The directional branches are based on asymmetric convolution designs [30].

C. Content-Adaptive Feature Reassembly

In the feature pyramid, nearest-neighbor upsampling enlarges feature maps by directly replicating feature values. This fixed operation does not adapt to the spatial structure of defects. To address this limitation, we introduce content-adaptive feature reassembly (CAFR), following the reassembly mechanism of Content-Aware ReAssembly of FEatures(CARAFE) [31]. CAFR predicts spatially varying kernels from the input features and uses them to aggregate local neighborhoods (Fig. 3), aiming to better preserve defect boundaries and weak responses during multi-scale fusion.

In DMFR-YOLO, CAFR replaces the upsampling operations at both top-down fusion stages, with upsampling factor $s = 2$ and reassembly kernel size $k = 5$. Its computation comprises two parallel paths followed by weighted aggregation: the weight-prediction path generates a normalized kernel for each output location, while the feature-extraction path obtains local neighborhoods from the original input. The predicted weights

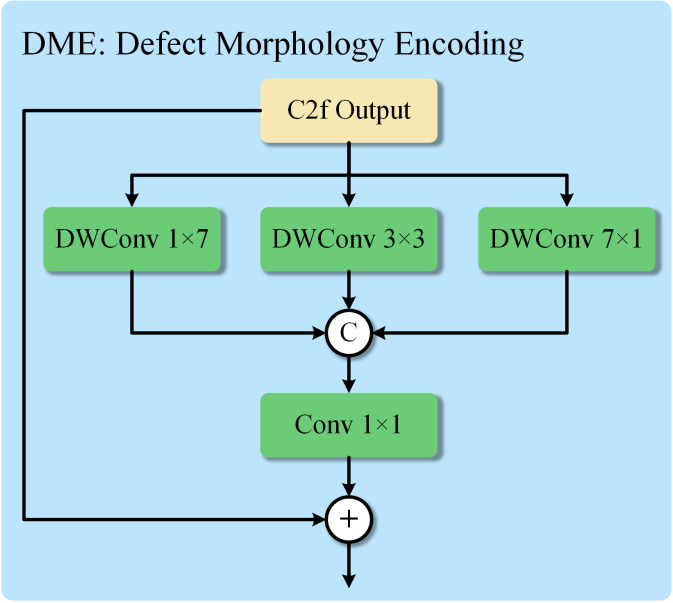

Fig. 2. DME structure. A C2f feature is processed by horizontal, vertical, and local depthwise branches, followed by channel concatenation, 1×1 fusion, and residual addition.

and source neighborhoods are aligned on the output grid before reassembly.

1) Reassembly Weight Prediction: Given an input feature map $X \in \mathbb{R}^{C\times H\times W}$, a 1×1 convolution first reduces its channel dimension:

$$X_c = E_{1\times 1}(X). \tag{6}$$

Here, $X_c \in \mathbb{R}^{D\times H\times W}$, where $D = \max(\lfloor C/4 \rfloor, 16)$ denotes the compressed channel count; $E_{1\times 1}$ includes batch normalization and SiLU activation. A linear 3×3 convolution then predicts the unnormalized reassembly weights:

$$K' = E_{3\times 3}(X_c). \tag{7}$$

The resulting tensor is $K' \in \mathbb{R}^{s^2k^2\times H\times W}$. For $s = 2$ and $k = 5$, this corresponds to 100 weight channels at each input location.

2) Reassembly Weight Generation: Pixel Shuffle rearranges the predicted weights to match the spatial resolution of the upsampled feature map. With $s = 2$, the 100 values predicted at each input position are distributed among its four corresponding output positions, giving 25 weights to each. For each output position (i, j), Softmax normalizes the corresponding k^2 weights:

$$K_{i,j} = \mathrm{softmax}([\mathrm{PS}(K')]_{:,i,j}). \tag{8}$$

With $k = 5$, these 25 normalized weights determine the contributions of the positions within a 5×5 input neighborhood.

3) Local Feature Extraction and Reassembly: In parallel with weight prediction, Unfold extracts a $k \times k$ neighborhood around each input location. Nearest-neighbor interpolation then replicates these neighborhoods onto the higher-resolution grid to align them with the predicted reassembly weights. For $s = 2$, the four output locations corresponding to an input location share the same neighborhood but use their own

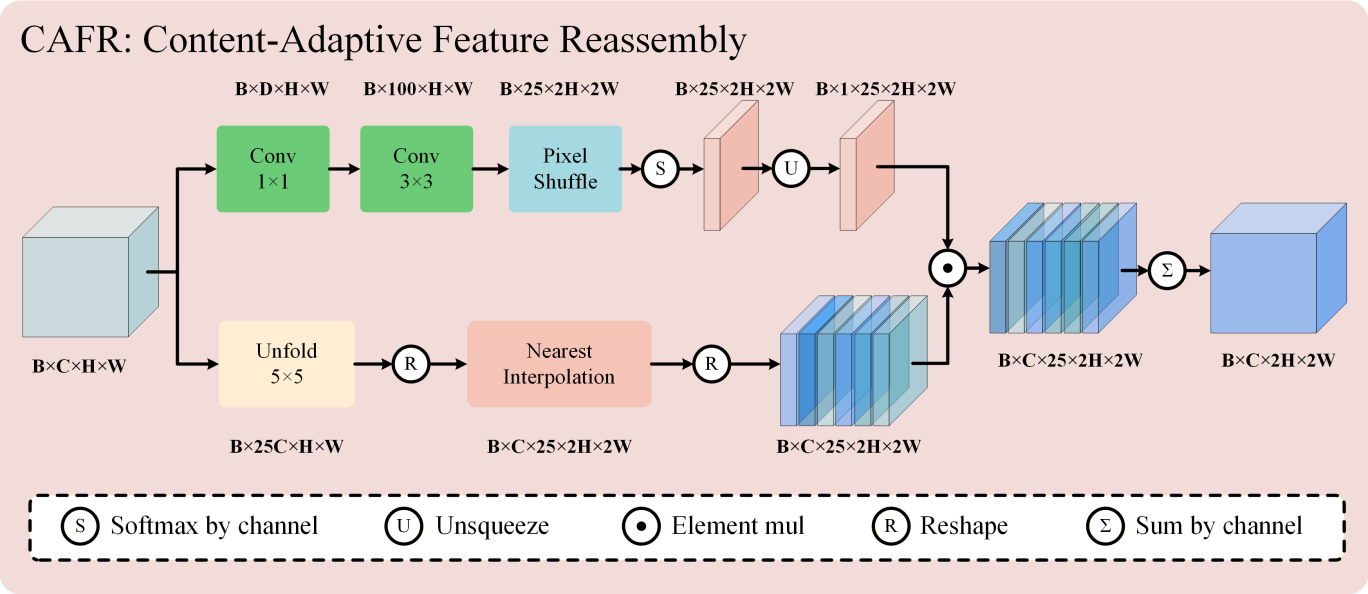

Fig. 3. CAFR structure. The upper path predicts location-specific 5×5 reassembly kernels, and the lower path unfolds local feature patches and replicates them to align with output locations. Weighted aggregation, rather than this alignment operation, produces the final $2\times$ upsampled feature.

predicted weights. Each output location aggregates the aligned neighborhood as

$$Y_{c,i,j} = \sum_{u=1}^{k^2} K_{i,j,u} P_{c,u,i,j}. \tag{9}$$

Here, $P \in \mathbb{R}^{C\times k^2\times sH\times sW}$ denotes the neighborhoods after replication and alignment, u indexes the k^2 positions within each neighborhood, and c denotes the feature channel. The predicted weights are shared across feature channels, producing $Y \in \mathbb{R}^{C\times sH\times sW}$. Nearest-neighbor interpolation is therefore used only for neighborhood alignment; the final output is obtained through content-adaptive weighted aggregation, allowing the four output locations to produce different feature responses.

By predicting location-specific reassembly weights, CAFR adaptively aggregates local features during upsampling, helping preserve defect-related information in multi-scale feature fusion.

D. Closed-Loop Detection System

Fig. 4 shows the installation of our defect detection system on a circular knitting machine production line. After the operator starts the circular knitting machine, the Hall sensor detects rotation and signals the RK3566 edge device to switch on the LED light and begin detection from the camera video stream. When a defect is detected, the device sends a stop signal to the machine controller through a wired connection to interrupt further defect propagation. The defect image and detection result are displayed locally for operator review and troubleshooting. The operator resolves the problem and restarts the machine, completing the operational cycle.

To enable on-device inference, the trained model is converted from PyTorch to ONNX and then to an INT8 RKNN model. The trained model is exported with a fixed 640×640 input and converted using RKNN-Toolkit2 [32], with post-training INT8 quantization for efficient NPU execution [33]. An Android application executes the RKNN model on the RK3566 edge device.

The cloud platform receives defect records and device states for centralized monitoring, defect statistics, and production-quality analysis. Its interface supports device-state visualization, remote configuration, and review of uploaded defect images (Fig. 5).

III. Experimental Setup

A. Dataset

Images were collected during operation of a real circular-knitting production line. A global-shutter industrial camera captures 1280×1024 frames approximately 10 cm from the fabric, with a field of view of approximately 16×14 cm. We selected images with visible defects and manually annotated their bounding boxes. The dataset includes three defect categories: horizontal defects (*row*), vertical defects (*col*), and holes (*hole*).

The dataset contains 403 images with 468 annotated defect instances, divided into training, validation, and test sets of 282, 40, and 81 images, respectively, as summarized in Table I. The validation set is used for checkpoint selection, while performance is reported on the test set containing 91 labeled defects.

TABLE I
IMAGE-LEVEL SPLIT AND DEFECT-INSTANCE COUNTS IN THE REAL PRODUCTION-LINE DATASET.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

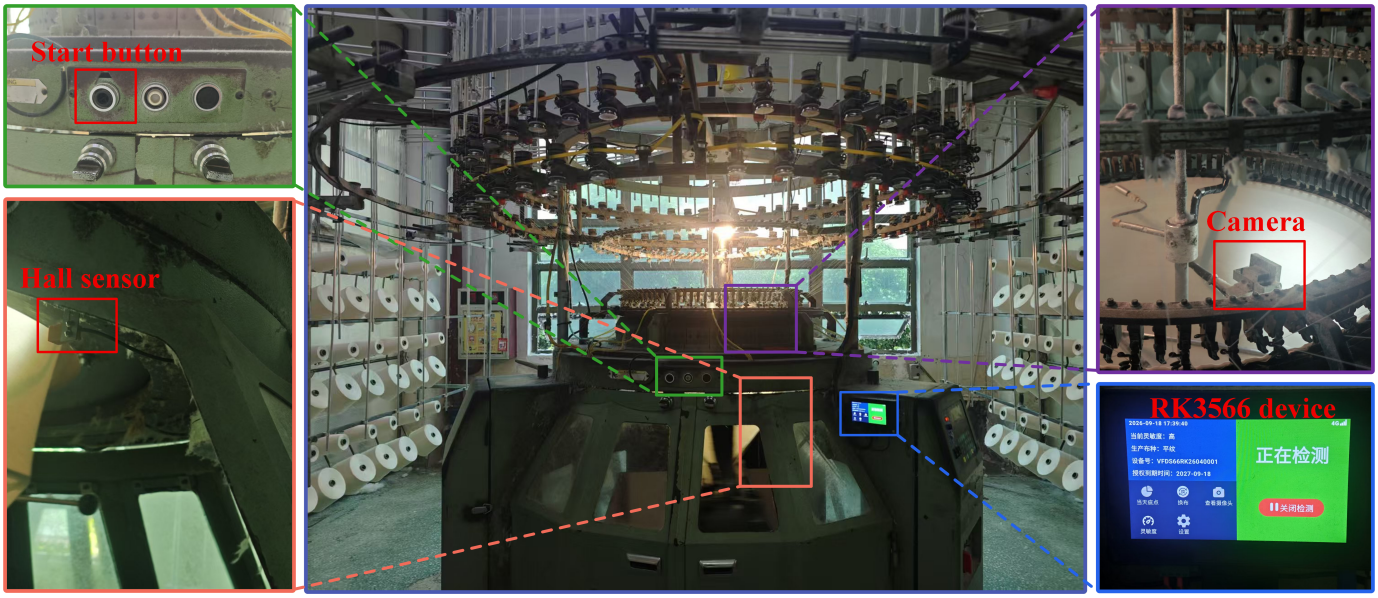

Fig. 4. Actual deployment on the circular knitting machine. The insets highlight the start button, Hall sensor, industrial camera, and RK3566 edge device, respectively.

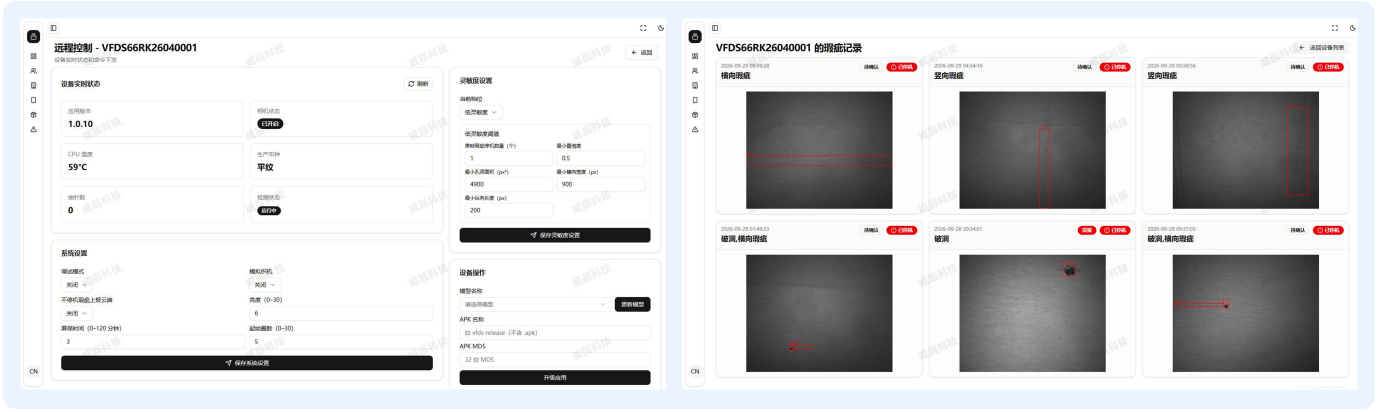

Fig. 5. IoT cloud interface: device-state monitoring and remote configuration (left), and uploaded defect records (right). Cloud functions support centralized supervision and quality analysis.

B. Implementation Details

Table II summarizes the training and deployment settings. DMFR-YOLO and YOLOv8n are initialized using the pre-trained yolov8n.pt weights, while newly added layers use default initialization. Other comparison models use the available pretrained checkpoints in their respective implementations. The remaining Ultralytics training settings follow the framework defaults.

The comparisons include lightweight YOLO models [24], [34], DETR [35], Deformable DETR [36], Faster R-CNN [37], and an R18 model from the RT-DETRv2 implementation [38], denoted RT-DETR-R18. All models are evaluated on the same test set using their respective frameworks and evaluation settings.

C. Evaluation Metrics

We report COCO average precision (AP) across ten IoU thresholds and AP at IoU 0.50 (AP50) [39]. For a predicted box B_p and a ground-truth box B_g, intersection over union is

TABLE II
IMPLEMENTATION PROTOCOL. TRAINING SETTINGS DESCRIBE THE ULTRALYTICS EXPERIMENTS; DEPLOYMENT SETTINGS DESCRIBE THE INSTALLED EDGE DEVICE.

Category	Parameter	Specification
1. Training setup	Hardware	Intel i7-13700KF; NVIDIA RTX 3090 (24 GB)
	Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
2. Deployment setup	Hardware	Rockchip RK3566
	System	Android 13
	Quantization	INT8
3. Data protocol	Input resolution	Fixed at 640×640
4. Training dynamics	Batch	16
	Maximum epochs	300
	Early-stopping patience	80 epochs

TABLE III
DETECTION RESULTS ON THE TEST SET. BOLD AND UNDERLINED VALUES INDICATE THE HIGHEST AND SECOND-HIGHEST REPORTED SCORES, RESPECTIVELY.

Model	Params (M)	FLOPs (G)	AP50 (%)	AP (%)
Lightweight YOLO Detectors				
YOLOv5n	2.50	7.10	93.1	58.4
YOLO11n	2.58	6.40	95.0	59.6
YOLO12n	2.56	7.30	94.3	60.9
YOLO26n	2.38	5.30	96.5	61.4
Transformer Detectors				
RT-DETR-R18	20.09	60.72	95.2	**67.7**
DETR-R50	41.28	77.25	96.4	53.6
Deformable DETR-R50	39.82	158.66	<u>96.8</u>	64.3
Two-Stage Detector				
Faster R-CNN-R50	41.13	181.45	96.1	64.7
Baseline				
YOLOv8n	3.01	8.10	94.1	59.3
Ours				
DMFR-YOLO	3.39	8.90	**96.9**	<u>66.3</u>

$\mathrm{IoU}(B_p, B_g) = |B_p \cap B_g|/|B_p \cup B_g|$. Let $\mathrm{AP}_c(t)$ denote the area under the interpolated precision–recall curve for class c at IoU threshold t, expressed on $[0, 1]$. For $N_c = 3$ classes, the percentage-valued metrics are

$$\mathrm{AP} = \frac{100}{10N_c} \sum_{c=1}^{N_c} \sum_{t \in \mathcal{T}} \mathrm{AP}_c(t),$$
$$\mathrm{AP50} = \frac{100}{N_c} \sum_{c=1}^{N_c} \mathrm{AP}_c(0.50), \tag{10}$$

where $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$. AP therefore evaluates localization over increasingly strict overlap requirements, whereas AP50 uses a single, less restrictive threshold. Differences between percentage-valued AP scores are reported in percentage points.

Model parameters and FLOPs are measured using the respective frameworks with an input resolution of 640 × 640.

IV. RESULTS AND DISCUSSION

A. Comparison With Other Detectors

Table III presents the detection results on the test set. DMFR-YOLO achieves 66.3% AP and 96.9% AP50, improving the YOLOv8n baseline by 7.0 and 2.8 percentage points, respectively. These improvements are achieved with only 0.38 M additional parameters and approximately 0.8 G additional FLOPs. The larger improvement in AP indicates better detection performance under stricter IoU thresholds.

Among the evaluated lightweight YOLO detectors, DMFR-YOLO achieves the highest AP. Although RT-DETR-R18 attains a slightly higher AP of 67.7%, it requires 20.09 M parameters and 60.72 G FLOPs, compared with 3.39 M parameters and 8.90 G FLOPs for DMFR-YOLO. Fig. 6 further illustrates the favorable balance between detection accuracy and model complexity achieved by DMFR-YOLO.

Fig. 7 compares the detection results of different models on seven selected test images, highlighting differences in detection

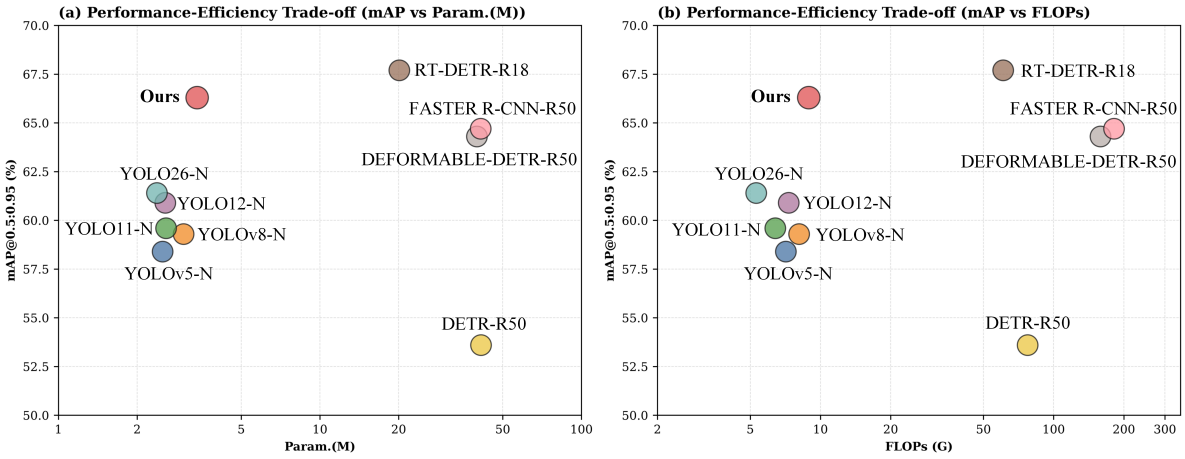

Fig. 6. Accuracy versus parameter count (left) and approximate FLOPs (right). Both horizontal axes are logarithmic. AP corresponds to the mAP@0.5:0.95 notation in the plot; Ours denotes DMFR-YOLO.

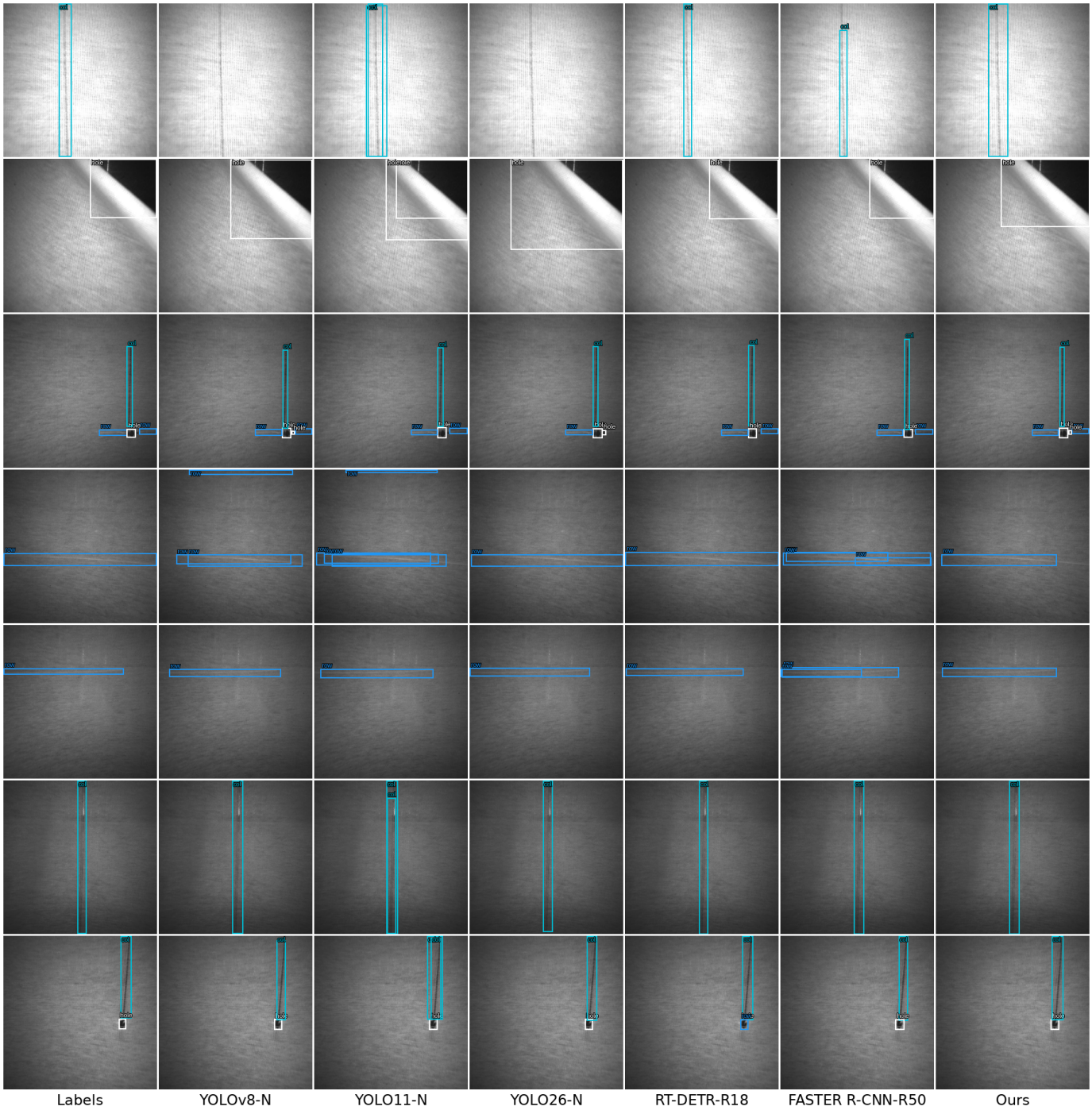

Fig. 7. Qualitative comparison of detection results on selected test images. From left to right, the columns show ground-truth annotations, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and DMFR-YOLO (Ours).

accuracy and bounding-box localization.

B. Component Ablation

Table IV evaluates the individual and combined effects of DME and CAFR on detection performance. Introducing DME increases AP from 59.3% to 64.1%, while CAFR alone improves AP to 60.7%. When both modules are incorporated, DMFR-YOLO achieves 66.3% AP and 96.9% AP50, outperforming either individual modification. These results demonstrate the complementary effects of the two modules. DME enhances the representation of defect morphologies in the backbone, while CAFR adaptively reconstructs these features during multi-scale fusion. Their combination further improves the overall detection performance.

Fig. 8 compares the feature visualizations of YOLOv8n and DMFR-YOLO DMFR-YOLO [40]. Compared with YOLOv8n, DMFR-YOLO exhibits more concentrated feature responses around elongated defects and holes, with activation regions more closely aligned with the defect shapes. These observations suggest that DMFR-YOLO provides a more focused

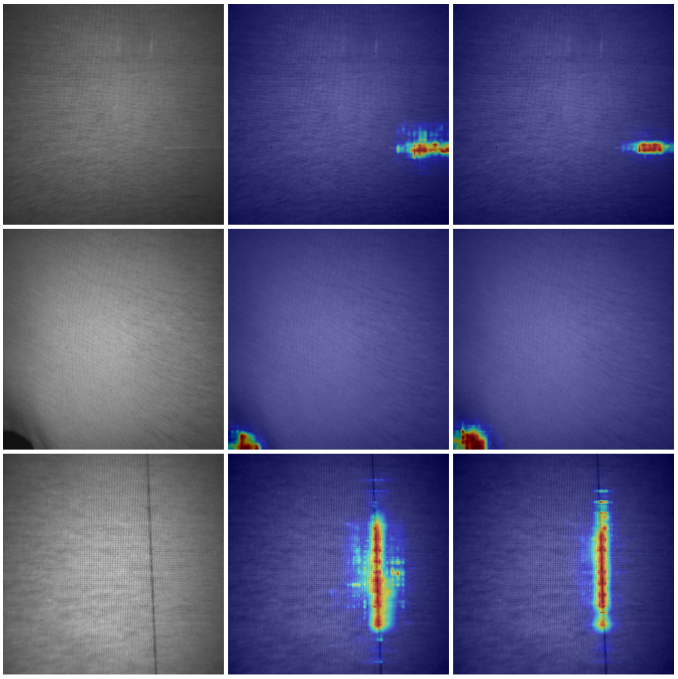

Fig. 8. Feature visualization of YOLOv8n and DMFR-YOLO. From left to right: original image, YOLOv8n, and DMFR-YOLO (Ours).

TABLE IV
COMPONENT ABLATION. PARAMETERS ARE IN MILLIONS, FLOPS IN BILLIONS, AND AP50/AP IN PERCENT.

Variant	DME	CAFR	Params	FLOPs	AP50	AP
YOLOv8n	–	–	3.01	8.1	94.1	59.3
+ DME	✓	–	3.28	8.8	94.2	64.1
+ CAFR	–	✓	3.11	8.2	94.9	60.7
DMFR-YOLO	✓	✓	3.39	8.9	**96.9**	**66.3**

representation of defect regions.

V. CONCLUSION

We have presented DMFR-YOLO and a production-line defect detection system for circular knitting machines. DMFR-YOLO couples defect morphology encoding with content-adaptive feature reconstruction for knitted-fabric defect detection. DME models horizontally elongated defects, vertically elongated defects, and localized hole-like defects in the backbone, while CAFR replaces fixed upsampling with CARAFE-based local reassembly in the top-down feature pyramid. On the production-line test set, DMFR-YOLO improves the YOLOv8n baseline from 59.3% to 66.3% AP and from 94.1% to 96.9% AP50, with 3.39 M parameters and approximately 8.9 G FLOPs. These results, together with the component ablation, support the complementary roles of defect morphology encoding and feature reassembly in improving overall localization performance while retaining a compact model scale.

The detector is deployed on an RK3566 edge device. Combined with Hall-sensor-based machine-state detection and wired stop control, the system performs local defect detection and intervention without depending on cloud communication. Detection results are displayed for on-site operator review and troubleshooting, while defect records and device states are uploaded to the cloud for equipment monitoring and production-quality analysis. This deployment connects the detector to local machine control, supporting not only offline defect detection but also in-process defect containment during circular-knitting production. Future work will focus on broader production conditions, long-term false-stop and missed-detection behavior, quantized-model accuracy, and finer-grained analysis of the proposed modules.

REFERENCES

[1] Cotton Incorporated, "Knit basics," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/knit-basics/. Accessed: Oct. 8, 2026.
[2] The Woolmark Company, "Circular knit technology," [Online]. Available: https://www.woolmark.com/industry/product-development/processing-innovations/circular-knitting-technology/. Accessed: Oct. 8, 2026.
[3] Cotton Incorporated, "Single and double knits," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/single-and-double-knits/. Accessed: Oct. 8, 2026.
[4] International Textile Manufacturers Federation, "Global textile machinery shipments shrunk in 2025, except for spinning," Jul. 2026, [Online]. Available: https://www.itmf.org/images/dl/press-releases/2026/2026-07-24-ITMSS-48-2025.pdf. Accessed: Oct. 10, 2026.
[5] S. Takeuchi, K. Nishioka, H. Uematsu, and S. Tanoue, "Research into development of the defect detection system for knitted fabric produced by the circular knitting machines by image analysis," *Journal of Textile Engineering*, vol. 64, no. 2, pp. 45–49, 2018.
[6] M. M. K. Akter, U. N. Haq, M. M. Islam, and M. A. Uddin, "Textile-apparel manufacturing and material waste management in the circular economy: A conceptual model to achieve sustainable development goal (SDG) 12 for bangladesh," *Cleaner Environmental Systems*, vol. 4, p. 100070, 2022.
[7] European Environment Agency, "Textiles," Circularity Metrics Lab, [Online]. Available: https://www.eea.europa.eu/en/circularity/sectoral-modules/textiles. Accessed: Oct. 8, 2026.
[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.
[9] S. Gao, M. Chu, and L. Zhang, "A detection network for small defects of steel surface based on YOLOv7," *Digital Signal Processing*, vol. 149, p. 104484, 2024.
[10] C. Li, A. Xu, Q. Zhang, and Y. Cai, "Steel surface defect detection method based on improved YOLOX," *IEEE Access*, vol. 12, pp. 37 643–37 652, 2024.
[11] X. Dong, C. Zhang, J. Wang, Y. Chen, and D. Wang, "Real-time detection of surface cracking defects for large-sized stamped parts," *Computers in Industry*, vol. 159, p. 104105, 2024.
[12] X. Kang and J. Li, "AYOLOv7-tiny: Towards efficient defect detection in solid color circular weft fabric," *Textile Research Journal*, vol. 94, no. 1–2, pp. 225–245, 2024.
[13] H. Yang, J. Jing, Z. Wang, Y. Huang, and S. Song, "YOLOV4-TinyS: A new convolutional neural architecture for real-time detection of fabric defects in edge devices," *Textile Research Journal*, vol. 94, no. 1–2, pp. 49–68, 2024.
[14] X. Luo, Z. Cheng, Q. Ni, R. Tao, and Y. Shi, "Defect detection algorithm for fabric based on deformable convolutional network," *Textile Research Journal*, vol. 93, no. 9–10, pp. 2342–2354, 2023.
[15] Z. Zhang, T. Huang, W. Zhang, Y. Yang, and Q. Yu, "DGHR-YOLO: Fabric defect detection based on high-level screening-feature pyramid networks and deformable convolution," *Nondestructive Testing and Evaluation*, vol. 40, no. 12, pp. 5919–5944, 2025.
[16] L. Zhou, B. Ma, Y. Dong, Z. Yin, and F. Lu, "DCFE-YOLO: A novel fabric defect detection method," *PLOS ONE*, vol. 20, no. 1, p. e0314525, 2025.
[17] X. Yue, Q. Wang, L. He, Y. Li, and D. Tang, "Research on tiny target detection technology of fabric defects based on improved YOLO," *Applied Sciences*, vol. 12, no. 13, p. 6823, 2022.
[18] K. Yu, W. Lyu, X. Yu, Q. Guo, W. Xu, and L. Zhang, "FA-YOLO: A high-precision and efficient method for fabric defect detection in textile industry," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107.A, no. 6, pp. 890–898, 2024.

[19] C. Chen, Q. Zhou, S. Li, D. Luo, and G. Tan, "Fabric defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, no. 3–4, pp. 235–251, 2025.

[20] T. Wang, N. Dai, X. Hu, K. Xu, and Y. Yuan, "SRSE-YOLO: An improved fabric defect detection method for YOLOv8," *Textile Research Journal*, 2025.

[21] Z. Gao, S. Zhang, J. Su, B. Li, J. Wang, and Z. Song, "Research on improved YOLOv8 fabric surface defect detection algorithm," *Textile Research Journal*, vol. 96, no. 17–18, 2026.

[22] S. Xu, C. Wan, W. Feng, J. Li, and J. Gao, "YOLO-DLL: An effective detection method for fabric defects," *Textile Research Journal*, 2025.

[23] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[24] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[25] Rockchip, "Rknn model zoo," [Online]. Available: https://github.com/airockchip/rknn_model_zoo, 2026, accessed: 2026-09-27.

[26] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2117–2125.

[27] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8759–8768.

[28] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[30] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1911–1920.

[31] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.

[32] Rockchip, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

[33] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[34] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025.

[35] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[36] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[37] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[38] W. Lv, Y. Zhao, Q. Chang, K. Huang, G. Wang, and Y. Liu, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," *arXiv preprint arXiv:2407.17140*, 2024.

[39] T.-Y. Lin, M. Maire, S. Belongie *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

[40] P.-T. Jiang, C.-B. Zhang, Q. Hou, M.-M. Cheng, and Y. Wei, "LayerCAM: Exploring hierarchical class activation maps for localization," *IEEE Transactions on Image Processing*, vol. 30, pp. 5875–5888, 2021.